

■ Design Distributed Cache

System Design Interview | Distributed Infrastructure

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	GET/SET/DELETE with TTL; Pub/sub; Atomic operations (INCR, compare-and-swap); LRU/LFU eviction
Non-Functional	Sub-millisecond latency; Horizontal scale; High availability; Consistency guarantees; Thundering herd protection
Scale	Terabytes of cached data; Millions of ops/sec; Distribution across dozens of nodes

■ Architecture Options

- **Client-Side Partitioning:** Client decides which cache node to use (consistent hashing); no coordination layer; used by Memcached clients
- **Proxy-Based:** Smart proxy (Twemproxy/Envoy) handles routing; client connects to proxy; can rebalance without client changes
- **Cluster Mode (Redis Cluster):** 16384 hash slots distributed across nodes; each node handles subset of slots; clients route directly with MOVED redirect
- **Replicated Caches:** Primary node + read replicas; writes to primary, reads from replicas; Redis Sentinel for automatic failover

■ Eviction Policies Deep Dive

- **LRU (Least Recently Used):** Evict key not accessed for longest time; good for temporal locality; Redis approximates with random sampling (not true LRU)
- **LFU (Least Frequently Used):** Evict key accessed least often; better for long-term hot keys; frequency counter per key
- **TTL-Based:** Keys expire after set duration; Redis uses lazy expiration + periodic background scan; memory freed lazily
- **Volatile-LRU:** LRU only among keys with TTL set; preserve non-expiring keys; useful when mixing temporary and permanent data

■ Thundering Herd & Hot Key Problems

- Thundering herd: cache miss on popular key → thousands of DB queries simultaneously → DB overloaded
- Solution 1 — Cache Locking: first request acquires distributed lock (Redis SET NX), fetches from DB, releases lock; others wait then read cache
- Solution 2 — Probabilistic Early Expiry: random early expiry for popular keys before TTL; background refresh; never truly expires
- Hot key (celeb problem): one key receiving millions of req/sec → single Redis node bottleneck; solution: local in-process cache layer + random suffix routing (key:rand(1-100))

■ Caching Patterns

- **Cache-Aside (Lazy Loading):** App checks cache; miss → read DB → populate cache; most common; risk: stale data if DB updated without cache invalidation
- **Write-Through:** Write to cache AND DB simultaneously; always consistent; higher write latency; Redis + DB atomic write
- **Write-Behind (Write-Back):** Write to cache only; async flush to DB; faster writes; risk of data loss on cache crash before flush
- **Read-Through:** App always reads from cache; cache handles DB fetch on miss; cache acts as single source of truth

■ Redis vs Memcached

Redis	Memcached
✓ Rich data types (sets, sorted sets, lists)	✓ Pure cache, simpler
✓ Persistence (RDB/AOF)	✓ Multi-threaded (better CPU util)
✓ Pub/sub, Lua scripting	✓ Consistent hashing by client
✓ Cluster mode native	✓ Lower memory overhead

■ Cache Invalidation Strategies

- TTL-based: simplest; eventual consistency; choose TTL based on data change frequency
- Event-driven invalidation: DB change → publish event → cache subscriber deletes/updates key; near real-time consistency
- Write-through: always update cache on write; consistent but adds write latency
- Cache-aside with versioning: store version in cache key (user:123:v5); increment version on update; old versions auto-expire